

DiP: A Scalable, Energy-Efficient Systolic Array for Matrix Multiplication Acceleration

Ahmed J. Abdelmaksoud[✉], *Member, IEEE*, Shady Agwa[✉], *Member, IEEE*,
and Themis Prodromakis[✉], *Senior Member, IEEE*

Abstract—Transformers are gaining increasing attention across Natural Language Processing (NLP) application domains due to their outstanding accuracy. However, these data-intensive models add significant performance demands to the existing computing architectures. Systolic array architectures, adopted by commercial AI computing platforms like Google TPUs, offer energy-efficient data reuse but face throughput and energy penalties due to input-output synchronization via First-In-First-Out (FIFO) buffers. This paper proposes a novel scalable systolic array architecture featuring Diagonal-Input and Permutated weight stationary (DiP) dataflow for matrix multiplication acceleration. The proposed architecture eliminates the synchronization FIFOs required by state-of-the-art weight stationary systolic arrays. Beyond the area, power, and energy savings achieved by eliminating these FIFOs, DiP architecture maximizes the computational resource utilization, achieving up to 50% throughput improvement over conventional weight stationary architectures. Analytical models are developed for both weight stationary and DiP architectures, including latency, throughput, time to full PEs utilization (TFPU), and FIFOs overhead. A comprehensive hardware design space exploration using 22nm commercial technology demonstrates DiP's scalability advantages, achieving up to a $2.02\times$ improvement in energy efficiency per area. Furthermore, DiP outperforms TPU-like architectures on transformer workloads from widely-used models, delivering energy improvement up to $1.81\times$ and latency improvement up to $1.49\times$. At a 64×64 size with 4096 PEs, DiP achieves a peak throughput of 8.192 TOPS with energy efficiency 9.548 TOPS/W.

Index Terms—Hardware acceleration, systolic arrays, spatial architectures, matrix multiplication, weight stationary.

I. INTRODUCTION

ARTIFICIAL Intelligence (AI) is increasingly dominating various application domains [1]. Natural Language Processing (NLP) is one of the crucial AI application domains receiving increasing attention nowadays [2], [3]. Thanks to Transformers, NLP tasks have been revolutionized by providing highly effective and scalable models for language understanding and generation [4]. The exceptional capabilities of these models have led to a transformer-driven transformation across numerous application domains, including Machine

Translation [5], Speech Recognition [6], Multimodal Applications [7], and Computer Vision [8].

However, transformers are data-intensive models that handle massive workloads compared to Deep Neural Networks (DNNs) and Convolutional Neural Networks (CNNs) [9]. Additionally, transformer models have been growing exponentially, evolving from the original vanilla Transformer model with around 65 million parameters to models with hundreds of billions of parameters [10], [11]. A clear example of the challenging level of scalability is GPT (Generative Pre-trained Transformer) model that incorporates billions of parameters, primarily involving matrix multiplications [12].

Conventional Von-Neumann architectures are struggling to meet these increasing performance demands due to the memory/data-movement bottleneck. Systolic arrays, introduced in 1982, are spatial architectures that designed to maximize the data utilization to mitigate the memory bottleneck. These architectures are receiving increased attention nowadays as a promising architecture for AI hardware acceleration [13]. Typically, a systolic array consists of a two-dimensional (2D) interconnected Processing Elements (PEs). The PE consists of basic arithmetic, mainly multiplication and accumulation, along with register units. Systolic arrays are spatial architectures that enhance local data utilization by increasing the number of computation operations per each memory access. Accordingly, input data circulates among PEs in a wave-like fashion, while the communication with the synchronization First-In-First-Out (FIFO)s occurs only at the boundary PEs. Moreover, the interconnection of the systolic array naturally facilitates data reusability among PEs, particularly during matrix multiplications [14].

Although many systolic arrays are used for hardware acceleration [15], [16], [17], [18], [19], [20], [21], different adopted dataflows require additional synchronization hardware, which limits the energy efficiency and constrains performance capabilities. Therefore, transformers with massive matrix multiplication workloads will face significant challenges in leveraging the systolic arrays for the next generation of AI hardware.

Tensor Processing Unit (TPU) is a well-known AI computing architectures introduced by Google to handle massive matrix multiplication workloads with higher performance and energy efficiency than CPUs and GPUs [22]. TPUs adopt weight stationary (WS) dataflow, which maximizes the data utilization of both weights and inputs. Google TPU V1 is designed primarily for inference, featuring a 256×256 systolic array optimized for 8-bit integer (INT8) operations,

Received 12 December 2024; revised 18 February 2025, 19 May 2025, and 14 July 2025; accepted 19 July 2025. This work was supported in part by the Engineering and Physical Sciences Research Council (EPSRC) Programme Grant “Functional Oxide Reconfigurable Technologies” (FORTE) Program under Grant EP/R024642/2 and in part by the RAEng Chair in Emerging Technologies under Grant CiET1819/2/93. This article was recommended by Associate Editor H. Cai. (*Corresponding author: Ahmed J. Abdelmaksoud.*)

The authors are with the Centre for Electronics Frontiers, Institute for Integrated Micro and Nano Systems, School of Engineering, The University of Edinburgh, EH9 3BF Edinburgh, U.K. (e-mail: a.j.abdelmaksoud@ed.ac.uk; shady.agwa@ed.ac.uk; t.prodromakis@ed.ac.uk).

Digital Object Identifier 10.1109/TCSI.2025.3591960

1549-8328 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

achieving a peak throughput of 92 TOPS [23]. TPU V2 shifted to mixed-precision training with a smaller 128×128 systolic array per core optimized for FP16 and bfloat16 operations, boosting throughput to 180 TeraFLOPS [24]. TPU V3 and V4 maintained the same array size but TPU V3 doubled the throughput to 420 TFLOPS per chip, aided by high memory bandwidth, while TPU v4 integrates four cores of 128×128 architecture, achieving up to 1 PFLOPS [25].

Meissa is a systolic architecture that separates multipliers from adders instead of integrating them into a unified array [26]. Similar to TPUs, it employs WS dataflow for the systolic array but eliminates input synchronization FIFOs to reduce overall latency. However, it has bulky adder trees per each column of the systolic array. These adder trees impose scalability limitations due to serious physical implementation challenges. As adder trees become larger, they require deeper pipelines to achieve higher frequencies. This increases the overall latency, area, and energy consumption. In addition, routing congestion presents another costly challenge, when all partial products from each PE column are delivered to the adder tree. Consequently, Meissa is not scalable to large $N \times N$ dimensions, which is critical for computation-intensive workload. Moreover, it still requires the output synchronization FIFOs, which add additional area, power, and energy penalty.

In this paper, we present a novel Diagonal-Input Permutated (DiP) weight stationary systolic array that overcomes the main challenges of the conventional WS systolic array architectures. The main contributions of this work are highlighted as follows:

- We introduce DiP, a novel scalable spatial architecture of $N \times N$ PEs for matrix multiplication acceleration. DiP maximizes PE utilization and energy efficiency, achieving a throughput improvement of up to $1.49\times$.
- The proposed architecture eliminates the input/output synchronization FIFOs in WS by implementing a new dataflow with diagonal-input movement and permutated weights.
- The analytical models are extracted for DiP and WS including throughput, latency, time to full PE utilization (TFPU), and register overhead for different systolic array sizes.
- Hardware design space exploration and implementation are presented for DiP and WS using commercial 22nm technology, offering up to $2.02\times$ energy efficiency per area improvement.
- DiP is evaluated using various transformer workloads from widely used models, outperforming TPU-like architectures and achieving up to $1.81\times$ energy efficiency improvement and up to $1.49\times$ latency improvement.

This paper is organized as follows; Section II discusses the systolic arrays background. Section III presents DiP architecture. Section IV shows hardware design space exploration, evaluation, and results. Finally, section V concludes the work.

II. SYSTOLIC ARRAYS BACKGROUND

Systolic arrays adopt one of the dataflows to control the data movement across the PEs. Each dataflow retains one of the input, output, or weight data to be stationary during

computations for the maximum duration to maximize the data reuse [27], [28]. The following dataflows are the most common in systolic array design:

- Weight Stationary (WS): weights are initially loaded to the PEs and kept stationary during processing. The input matrix and partial summations move among the PEs.
- Input Stationary (IS): input matrix is initially loaded to the systolic array, while weight matrix and partial summations move among the PEs.
- Output Stationary (OS): input and weight matrices move among the PEs, while the partial summations are accumulated inside PEs.
- Row Stationary (RS): it is proposed by Eyeriss [29]. It adopts spatial architecture that uses coarse-grained PEs with internal memories to store weights and inputs. Inputs are broadcasted diagonally across the PEs, while weight matrix broadcasted horizontally, and partial summations move vertically.

The OS dataflow moves both input and weight matrices simultaneously, which effectively doubles the required memory bandwidth for the systolic array. In RS dataflow, data redundancy increases because copies of the data are loaded into multiple PEs. Additionally, the circulation of weights within each PE reduces energy efficiency. On the other hand, WS dataflow is widely used in many architectures, such as Google TPU, due to its scalability and flexibility in handling convolutions and matrix multiplication [23], [24], [25]. Moreover, it requires less memory bandwidth. Therefore, we focus on improving this dataflow.

A. WS Dataflow

The WS dataflow is widely used in many architectures where weights are initially loaded to the PEs, while the input matrix circulates between them in a systolic fashion. This approach mitigates the memory bottleneck by reducing memory accesses and increasing data reuse. However, WS dataflow requires input and output FIFOs to synchronize data for proper functionality. Figure 1 shows the architecture of WS systolic array consisting of $N \times N$ PEs and two FIFO groups for input/output synchronization. WS PE consists of 2-stage pipelined MAC unit and four enabled registers for input, weight, multiplier output and adder output. Control signals *wshift*, *pe_en*, *mul_en*, and *adder_en* manage the PE operations. Specifically, *wshift* enables the weight register, while *pe_en* enables the input register. *mul_en* and *adder_en* selectively enable their respective registers only during active computation cycles, reducing power consumption during inactive cycles. *wshift* is shared across each PE row, while *pe_en*, *mul_en*, and *adder_en* are shared across each PE column. The input FIFO group consists of series of FIFOs with incrementally increasing depth starting from one element in the second row to $N - 1$ elements in the last row. The output FIFO group is structured similarly to the input FIFO group, but the FIFO depths decrease from $N - 1$ elements to one element moving from left to right across the systolic array columns.

The input FIFOs map the input matrix to the PEs in diagonal wave fashion, while the output FIFOs synchronize

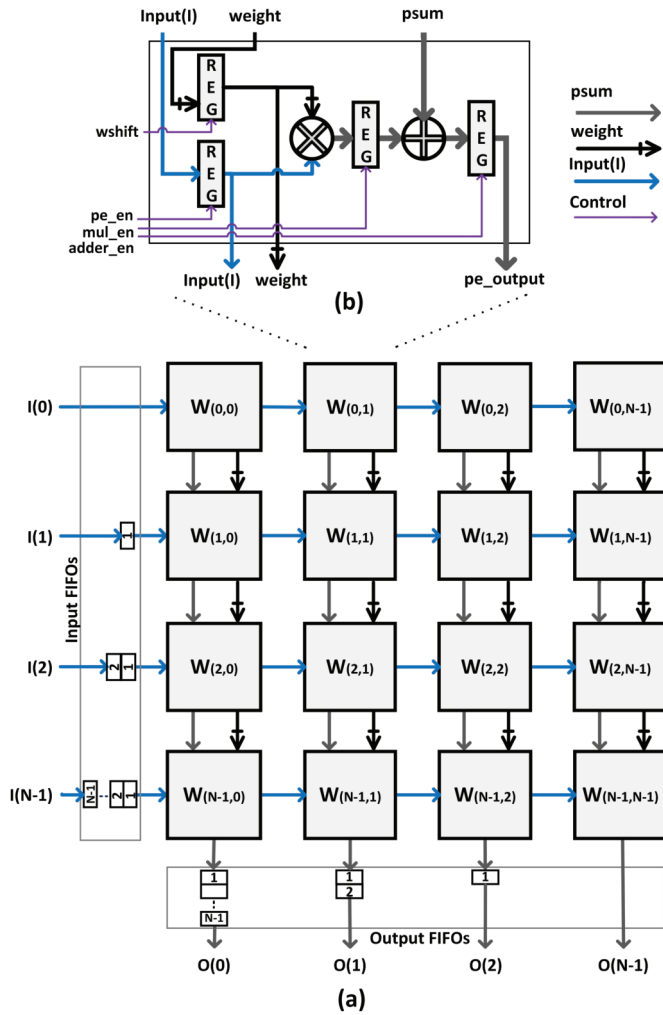


Fig. 1. (a) Top-level schematic for the WS systolic array with $N \times N$ PEs and two input/output FIFO groups. The weights (black crossed buses) are loaded vertically, and psums (grey buses) are accumulated along the columns. (b) WS PE block diagram, consisting of a 2-stage pipelined MAC unit and four enabled registers. Control signals $wshift$, pe_en , mul_en , and $adder_en$ are used for operations control. $wshift$ is shared across each PE row, while pe_en , mul_en , and $adder_en$ are shared across each PE column.

partial summations along each column [27]. In WS, the input matrix and partial summations (psums) are pipelined to ensure proper processing and improve data reuse. The PE in WS is fine-grained, unlike the coarse-grained PE in Eyeriss, which broadcasts weights and input matrices to the array and stores them in internal memories [29]. Weights are loaded vertically, while the input matrix is shifted horizontally, while the output is shifted to the output FIFO group. Using synchronization FIFOs add significant overheads in terms of area, power, energy consumption, and latency during matrix multiplication. Moreover, the computations propagate as a diagonal wave from the top-left corner to the bottom-right corner of the systolic array. This substantially reduces overall PE utilization resulting in degraded performance and increased latency. As a result, FIFO-based systolic arrays suffer from lower energy efficiency, reduced PE utilization, higher latency, and decreased throughput.

The analytical modeling of the WS systolic array is studied for latency, throughput, TFPU, and FIFOs overhead, providing insights into dataflow performance. For the WS latency analytical model, as shown in (1), WS requires $3N + S - 3$ cycles to complete the processing, where N represents the number of rows/columns per WS systolic array, and S is the number of pipeline stages per Multiply-Accumulate (MAC) unit, which equals one for 1-stage pipelined MAC, and two for 2-stage pipelined MAC. The throughput, as indicated in (2), is calculated as the ratio of total operations to WS latency. Regarding register overhead, WS systolic array uses two FIFO groups for input and output synchronization. Each group consists of $N-1$ FIFOs, as shown in Fig. 1. Additionally, each FIFO group includes $N(N-1)/2$ registers. Consequently, the total register overhead for a typical WS systolic array is calculated as shown in (3). TFPU is another metric introduced to calculate the required number of cycles to reach full utilization of PEs. This metric shows the overhead when the input matrix is initially loaded to the PEs. TFPU is calculated, as shown in (4), where it takes $2N - 1$ cycles for WS to reach full PEs utilization.

$$\text{Latency for WS} = 3N + S - 3 \quad (1)$$

$$\text{Throughput for WS} = \frac{2N^3}{3N + S - 3} \quad (2)$$

$$\text{Registers overhead for WS} = N(N-1) \quad (3)$$

$$\text{TFPU for WS} = 2N - 1 \quad (4)$$

III. DiP ARCHITECTURE

In this section, we discuss DiP architecture, DiP dataflow, and DiP analytical models. Then, we compare the analytical models for DiP and WS.

A. DiP Architecture

DiP is a scalable spatial architecture consisting of $N \times N$ PEs, designed to accelerate matrix multiplication computations, as shown in Fig. 2(a). The input matrix moves diagonally across the PEs, passing from one PE row to the next. The boundary PEs are diagonally connected, with the registered inputs of the leftmost PE column are connected to the inputs of the rightmost PE column in the subsequent row. The weight matrix is loaded vertically, and psums are accumulated vertically. Figure 2(b) shows the architecture of each PE.

The proposed PE is similar to WS PE, utilizing a 2-stage pipelined MAC unit and four enabled registers for input, weight, multiplier output and adder output. Control signals $wshift$, pe_en , mul_en , and $adder_en$ manage the PE operations. Specifically, $wshift$ enables the weight register, while pe_en enables the input register. mul_en and $adder_en$ selectively enable their respective registers only during active computation cycles, reducing power consumption during inactive cycles. However in DiP, $wshift$, pe_en , mul_en , and $adder_en$ are shared across each PE row.

B. DiP Dataflow

The proposed DiP architecture adopts a novel dataflow to control the movement of inputs, weights, and partial summations across the whole systolic array. DiP dataflow relies on

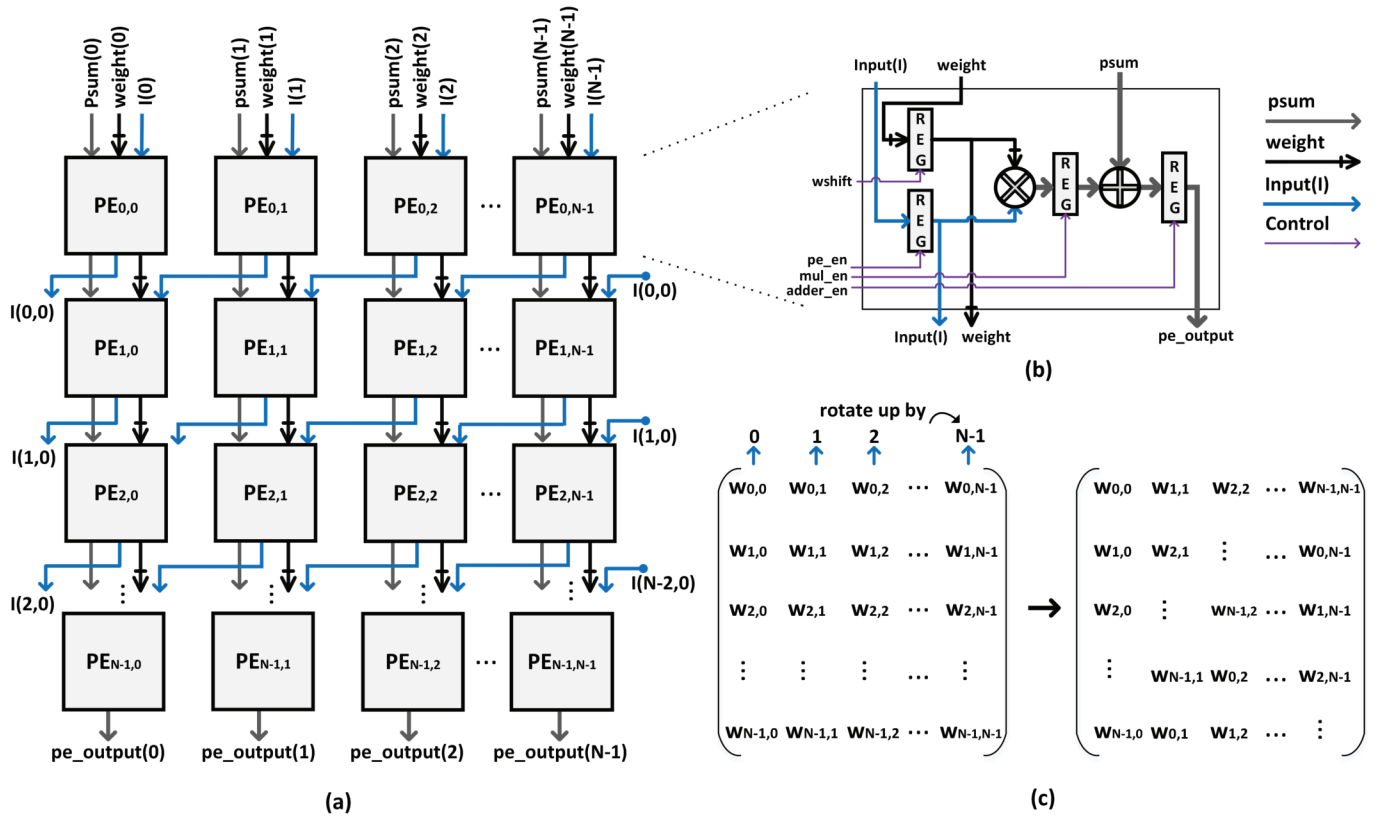


Fig. 2. (a) $N \times N$ DiP systolic array architecture, inputs(I) move diagonally across PE rows, transitioning from one row to the next. The boundary PEs are diagonally connected, so that the registered inputs from the leftmost PE column feed into the inputs of the rightmost PE column in the subsequent row. Weights are loaded vertically, and psums are accumulated vertically along the columns as well. (b) PE block diagram, consisting of a 2-stage pipelined MAC unit and four enabled registers. Control signals $wshift$, pe_en , mul_en , and $adder_en$ are used for operations control. $wshift$ is shared between all systolic array PEs, while pe_en , mul_en , and $adder_en$ are shared across each PE row. (c) General weight matrix permutation for DiP dataflow. The weight matrix is permuted by shifting and rotating each column by its column index.

Algorithm 1 Matrix Permutation for DiP Dataflow

Input: *matrix*

Output: *permuted_matrix*

for $i \leftarrow 0$ to cols do

for $j \leftarrow 0$ to rows do

$permuted_matrix[j][i] = matrix[(j+i) \% rows][i]$

end for

end for

two major upgrades compared to WS dataflow, the diagonal movement of input matrix, and the weight matrix permutation. Firstly, the input matrix moves diagonally across PE rows, as shown in Fig. 2(a). Secondly, the proposed dataflow permutes the weights by shifting and rotating each column by its column index, as shown in Fig. 2(c). The weights permutation is initially prepared on software level, as shown in Algorithm 1. For each column, it iterates over all rows, assigning each element in the *permuted_matrix* based on its row and column index. The permutation is performed at the software level when the second matrix is a weight matrix. Alternatively, it is executed at run-time by efficiently re-scheduling memory access across multi-bank memories with almost zero overhead. The proposed dataflow removes the need for input and output synchronization FIFOs typically required by

conventional WS systolic arrays. Additionally, it enhances throughput and PE utilization while reducing chip area and latency.

Figure 3 shows a complete example for 3×3 DiP systolic array. It consists of three rows: a) Row-0: $PE_{00}, PE_{01}, PE_{02}$, b) Row-1: $PE_{10}, PE_{11}, PE_{12}$, c) Row-2: $PE_{20}, PE_{21}, PE_{22}$. The PE array is diagonally connected, as shown in Fig. 3(a). The leftmost PE in each PE row is connected to the rightmost PE in the next PE row. The weight matrix is initially permuted to be prepared for weights loading. Each column is shifted and rotated by its column index, as shown in Fig. 3(b). The weights are initially loaded, row by row, to the systolic array, as shown in Fig. 3 from Cycle -2 to Cycle 0. The loading of the last weight row and the first input matrix are performed in parallel at Cycle 0. The input data is loaded in parallel to Row-0 including inputs for PE_{00}, PE_{01} , and PE_{02} . After accomplishing the required computations by Row-0, the input data is shifted diagonally from PE_{00} to PE_{12} and from PE_{01} to PE_{10} and from PE_{02} to PE_{11} . Then after accomplishing the required computations by Row-1, the input data is shifted diagonally again from PE_{10} to PE_{22} and from PE_{11} to PE_{20} and from PE_{12} to PE_{21} . The processing starts from Cycle 1 to Cycle 5, while the first output row becomes ready at Cycle 3, and the last output row becomes ready at Cycle 5. The processing goes as follows:

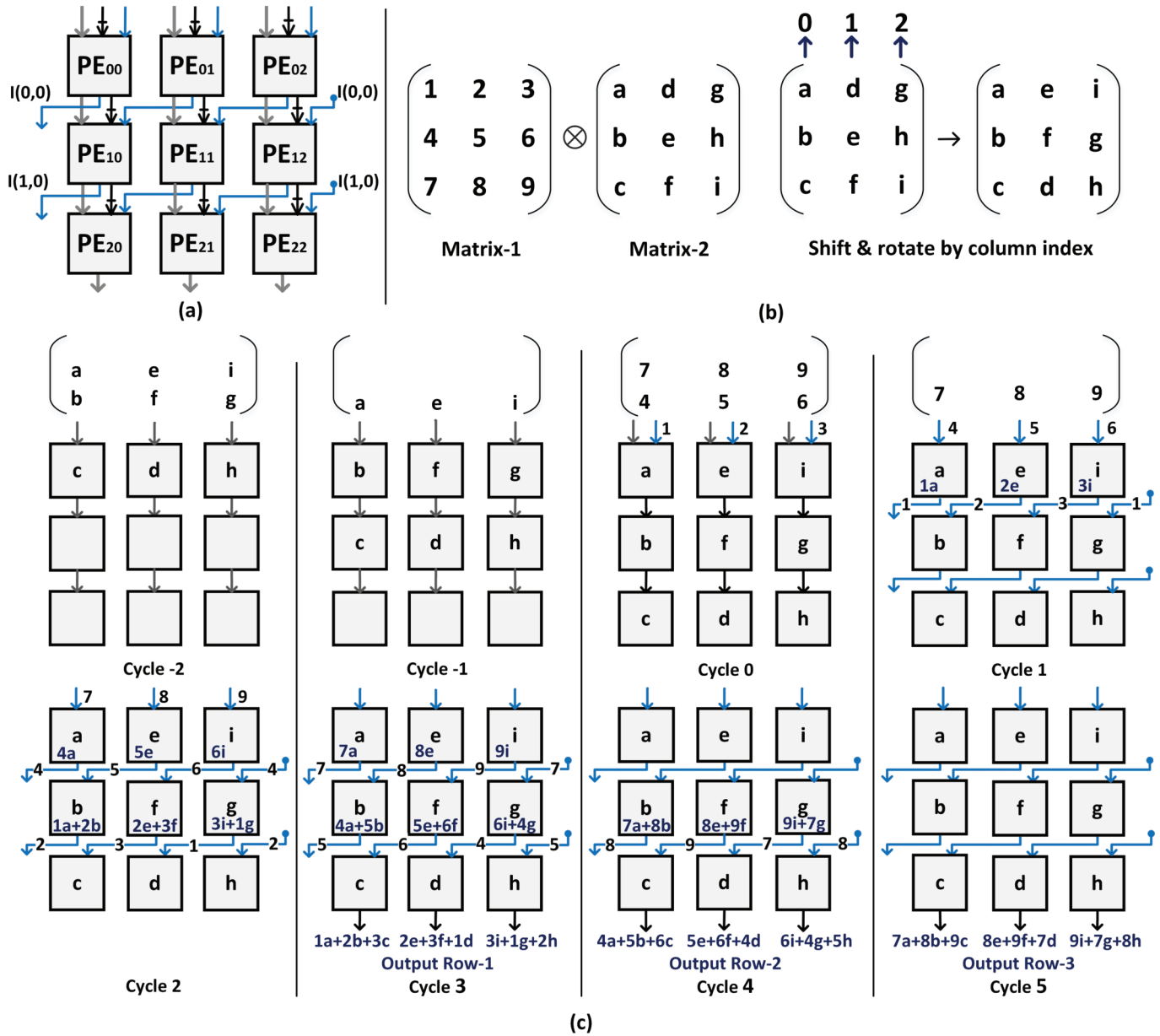


Fig. 3. Example for 3×3 DiP systolic array (a) shows the diagonal input connections for 3×3 DiP, (b) shows 3×3 DiP weight matrix permutation by shifting and rotating each column by its column index, and (c) shows the processing flow for 3×3 DiP example cycle by cycle. Cycles $(-2, -1, 0)$ are dedicated to weight matrix loading, while Cycle 0 involves loading the last row of the weight matrix and the first row of the input matrix. Cycles from Cycle 1 to Cycle 5 are allocated for matrix multiplication processing, while the first output row becomes ready at Cycle 3.

- **Cycle -2:** The last row of weight matrix (c, d, h) is loaded to the first PE row.
- **Cycle -1:** The weights matrix row (c, d, h) is shifted to the second PE row, and new weights row (b, f, g) is loaded to the first PE row.
- **Cycle 0:** The weight matrix row (c, d, h) is shifted to the last PE row, the weights in the first PE row (b, f, g) are shifted to the second PE row, and new weights row (a, e, i) and the first input matrix row $(1, 2, 3)$ are loaded to the first PE row, simultaneously.
- **Cycle 1:** The first PE row shifts the partial summations $(1a, 2e, 3i)$ to the second row. Using the diagonal connections, the input matrix row $(1, 2, 3)$ is permuted to $(2, 3, 1)$, and loaded to the second PE row at the same cycle.
- **Cycle 2:** The first PE row shifts the partial summations $(4a, 5e, 6i)$ to the second row, and the input matrix row $(4, 5, 6)$ is permuted to $(5, 6, 4)$ and loaded to the second PE row. Similarly, The second PE row shift the partial summations $(1a+2b, 2e+3f, 3i+1g)$ to the third row, and the input matrix row $(2, 3, 1)$ is permuted to $(3, 1, 2)$ and loaded to the third PE row.
- **Cycle 3:** The first PE row shift the partial summations $(7a, 8e, 9i)$ to the second row, and the input matrix row $(7, 8, 9)$ is permuted to $(8, 9, 7)$ and loaded to the second PE row. Similarly, The second PE row shift the

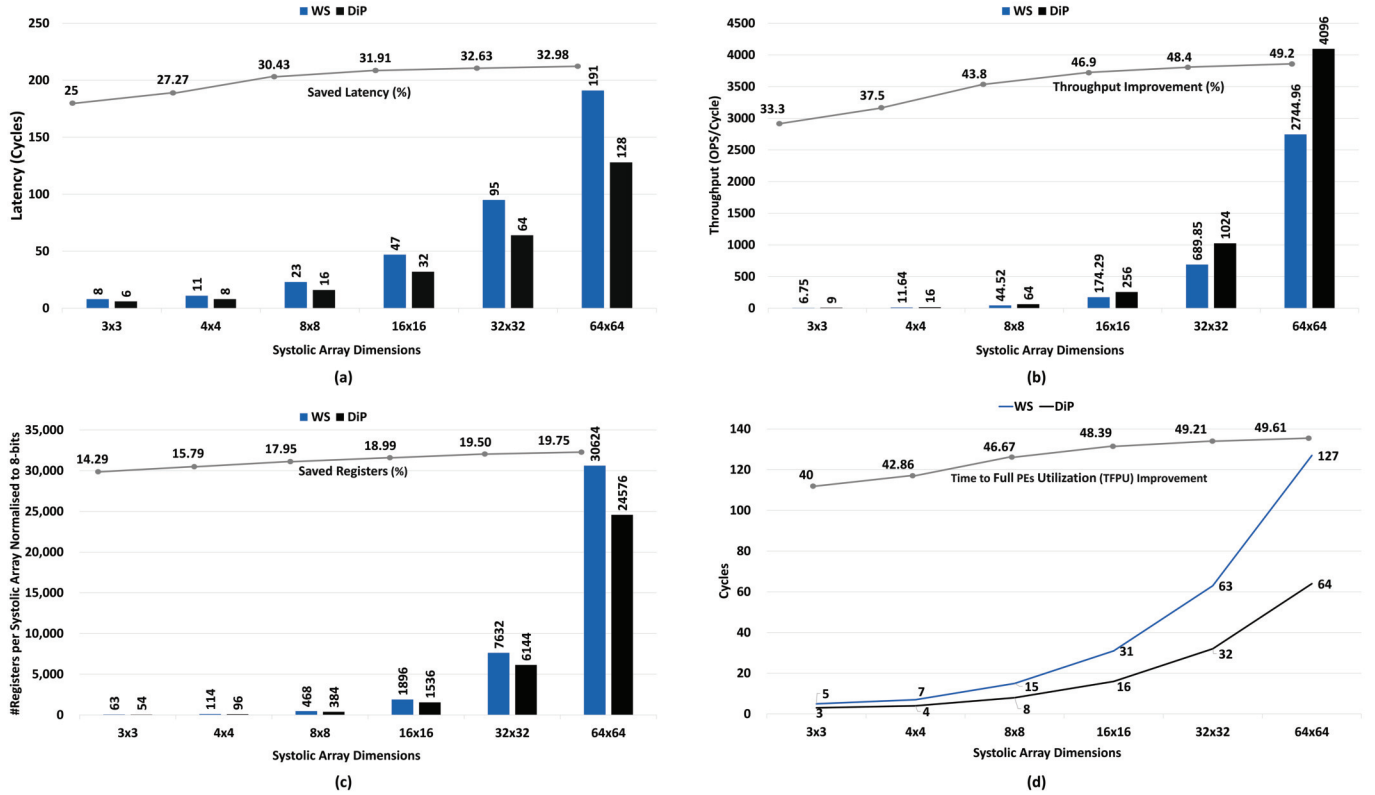


Fig. 4. Analytical modeling results comparing WS and DiP in terms of latency, throughput, register usage, and TFPU. (a) Latency (per single tile processing) for WS and DiP systolic arrays, with the grey curve indicating the percentage of latency savings of DiP over WS. (b) Throughput, measured in operations per cycle (OPS/Cycle), for WS and DiP, with the grey curve showing the percentage throughput improvement of DiP compared to WS. (c) Register usage for DiP compared to WS, normalized to 8-bit (baseline datawidth), with the grey curve indicating the percentage of register savings. (d) TFPU, representing the time required to activate all PEs, with the grey curve showing the percentage TFPU improvement. WS and DiP are represented in blue and black, respectively.

partial summations ($4a+5b$, $5e+6f$, $6i+4g$) to the third row, and the input matrix row (5, 6, 4) is permuted to (6, 4, 5) and loaded to the third PE row. In addition, The first output row ($1a+2b+3c$, $2e+3f+1d$, $3i+1g+2h$) is shifted out.

- **Cycle 4:** The first PE row becomes idle, unless more input rows are loaded, and the partial summations ($7a+8b$, $8e+9f$, $9i+7g$) are shifted from the second PE row to the third row. In addition, the input matrix row (8, 9, 7) is permuted to (9, 7, 8), and loaded to the third PE row. In addition, the second output row ($4a+5b+6c$, $5e+6f+4d$, $6i+4g+5h$) is shifted out.
- **Cycle 5:** The first and second PE rows are idle, and the third output row ($7a+8b+9c$, $8e+9f+7d$, $9i+7g+8h$) is shifted out.

Meanwhile, additional associated input tiles to the loaded weight tile are scheduled for larger input matrix, and the processing continues till the end of the workload.

C. DiP Analytical Model

The analytical models for DiP are studied for latency and throughput. Regarding DiP latency, DiP systolic array requires $2N+S-2$ cycles for processing, where N is the number of rows/columns per DiP systolic array, and S is the number of pipelined stages per MAC unit. As a result, the latency is $2N-1$ cycles for 1-stage pipelined PE, and $2N$ cycles for 2-stage

pipelined PE, as shown in (5). The throughput is calculated as the number of operations (multiplications and additions) divided by the latency, as shown in (6). TFPU is calculated, as shown in (7), where it takes N cycles to reach full PEs utilization, outperforming WS by $N-1$ cycles. Moreover, The registers overhead is eliminated by removing the input/output synchronization FIFOs.

$$\text{Latency for DiP} = 2N + S - 2 \quad (5)$$

$$\text{Throughput for DiP} = \frac{2N^3}{2N + S - 2} \quad (6)$$

$$\text{TFPU for DiP} = N \quad (7)$$

D. DiP/WSSA Analytical Comparison

The scalability of systolic arrays is important to meet the acceleration requirements. The proposed systolic array is gradually scaled up from 3×3 to 64×64 with sizes (3×3 , 4×4 , 8×8 , 16×16 , 32×32 , 64×64). An analytical comparison of DiP and WS is presented, evaluating throughput, latency, register savings, and TFPU across different systolic array sizes. Figure 4(a) shows the latency for DiP compared to WS, with the percentage of latency savings calculated as the difference between WS and DiP latencies, divided by WS latency. The saved percentage starts at 28% for a 3×3 systolic array and reaches 33% for a 64×64 systolic array.

TABLE I

COMPARISON OF AREA, POWER CONSUMPTION, AND IMPROVEMENTS IN THROUGHPUT, POWER CONSUMPTION, AREA, AND OVERALL PERFORMANCE OF DiP COMPARED TO WS AT DIFFERENT SIZES USING COMMERCIAL 22NM TECHNOLOGY AT FREQUENCY OF 1GHZ

Size	Area (mm ²)		Area Improvement (×)	Power Consumption (mW)		Power Consumption Improvement (×)	Throughput Improvement (×)	Overall Improvement [*] (×)
	WS	DiP		WS	DiP			
4×4	0.0052	0.0049	1.06	4.168	3.582	1.16	1.38	1.70
8×8	0.0187	0.0174	1.08	16.2	13.72	1.18	1.44	1.84
16×16	0.0712	0.0654	1.09	64.28	53.63	1.20	1.47	1.92
32×32	0.2750	0.253	1.09	264.2	211.5	1.25	1.48	2.02
64×64	1.085	1.012	1.07	1041	857.8	1.21	1.49	1.93

*Overall improvement represents energy efficiency per area.

In addition, The throughput for both the DiP and WS systolic arrays is compared, as shown in Fig. 4(b). The throughput improvement is calculated as the ratio between DiP to WS throughput. The improvement ratio starts at 33.3% for a 3×3 systolic array and reaches 49.2% for a 64×64 systolic array. The proposed architecture significantly increases the PEs utilization. Thus, it outperforms the conventional WS counterparts in terms of throughput by up to 50%.

Moreover, Fig. 4(c) shows the percentage of saved registers as another design improvement of DiP compared to the WS systolic array. By eliminating input/output FIFOs, the percentage of saved registers reaches up to 20% for a 64×64 systolic array. The saved registers is calculated as the difference between WS and DiP used registers, divided by the number of registers used by WS. The registers of WS systolic array is distributed between input synchronization FIFOs, Output synchronization FIFOs, and internal PE registers. In contrast, the proposed diagonal input systolic array relies solely on internal PE registers, eliminating the need for input/output FIFOs. The represented number of registers are normalized to 8-bit as the baseline bandwidth.

TFPU calculates the required number of cycles to reach full utilization of PEs. This metric shows the overhead when the inputs are initially loaded, particularly for large matrix-matrix multiplication. Figure 4(d) shows TFPU for WS and DiP systolic arrays. The proposed DiP rapidly utilizes all PEs row by row, whereas WS gradually activates PEs in a diagonal pattern, starting from the top-left and moving to the bottom-right. Consequently, DiP outperforms WS, requiring only almost half the time of WS to fully utilize the entire systolic array.

IV. EVALUATION & RESULTS

This section explores the hardware design space for the proposed DiP architecture, followed by benchmarking DiP using transformers and evaluating it against TPU-like architecture across various transformer workloads. Finally, DiP is compared with existing accelerators in the literature.

A. Hardware Design Space Exploration

A hardware design space exploration is developed for DiP and WS at different sizes. Both designs are scaled from 4×4 to 64×64 with variants (4×4 , 8×8 , 16×16 , 32×32 , 64×64). A parameterized HDL design using Verilog is developed. Then, all designs are implemented from synthesizer to GDSII

TABLE II

MATRIX MULTIPLICATION DIMENSIONS FOR MHA AND FFN WORKLOADS IN TERMS OF SEQUENCE LENGTH (l), MODEL HIDDEN SIZE (d_{MODEL}), HEAD SIZE (d_k), FFN SIZE (d_{FFN}). INPUT MATRICES SIZES ARE $M \times N$ AND $N \times K$, WHILE OUTPUT MATRIX IS $M \times K$

Workload	Dimensions (M, N, K)
Input Projections, $[Q_i, K_i, V_i] = X \cdot [W_i^Q, W_i^K, W_i^V]$	$l \times d_{\text{model}} \times d_k$
$A_i = Q_i \cdot K_i^T$	$l \times d_k \times l$
Attention Scores, $S_i = \text{Softmax}(A_i / \sqrt{d_k})$	$l \times d_k \times l$
Attention, $\text{Attn}_i = S_i \cdot V_i$	$l \times l \times d_k$
Output Projection, $Y = \text{Concat}(\text{Attn}_1, \text{Attn}_2, \dots, \text{Attn}_h) \cdot W^O$	$l \times d_{\text{model}} \times d_{\text{model}}$
FFN1 Projection, $Z = \text{Non-Linear}(Y \cdot W_1 + b_1)$	$l \times d_{\text{model}} \times d_{\text{FFN}}$
FFN2 Projection, $F = Z \cdot W_2 + b_2$	$l \times d_{\text{FFN}} \times d_{\text{model}}$

using commercial 22nm technology at frequency of 1 GHz. Table I shows a comparison between WS and DiP at different sizes in terms of area and power consumption. Additionally, the improvements in throughput, power consumption, area, and overall improvement (energy efficiency per area) for different WS/DiP design space. DiP outperforms WS across all metrics, with overall performance from $1.7 \times$ to $2.02 \times$. At a size of 32×32 , DiP achieves $1.48 \times$ higher throughput than WS, $1.25 \times$ lower power consumption, and $1.09 \times$ smaller area footprint, resulting in a total improvement of $2.02 \times$. Additionally, at size of 64×64 , the throughput is improved by $1.49 \times$, power consumption is reduced by $1.21 \times$, and area is decreased by $1.07 \times$ compared to WS, with overall improvement $1.93 \times$.

B. Transformers Benchmarking

Transformer workloads are becoming increasingly massive and heavily dependent on matrix multiplication, especially in Multi-Head Attention (MHA) and Feed-Forward Networks (FFN). MHA, a core component of transformer models proposed by Vaswani et al., enables the capture of complex dependencies in data [10]. By leveraging multiple attention heads, MHA captures diverse representational subspaces, allowing the model to understand relationships across different perspectives simultaneously.

Table II presents the matrix multiplication workloads and their dimensions for MHA and FFN operations within transformer models, highlighting the relationship between input sequence length (l), model size (d_{model}), head size (d_k), and FFN size (d_{FFN}). The table offers insights into the computational requirements of transformer models, emphasizing how the sequence length and model size impact the matrix

operations in both the attention mechanism and the feed-forward network. In MHA, the input (X) is first projected in Queries (Q_i), Keys (K_i), and Values (V_i) per each head i using learned weight matrices. The attention scores (S_i) for each head are computed by taking the scaled dot product of Queries (Q_i) and transposed Keys (K_i), followed by a softmax normalization. The attention scores (S_i) are multiplied with Values (V_i), producing the attention output ($Attn_i$) for each head. The outputs from all heads are concatenated into a single matrix $Attn_{concat}$, which is finally projected back to the model's hidden dimension (d_{model}) using a learned output projection matrix (W^O). This process enables the model to capture information from multiple representation subspaces simultaneously, enhancing the model's ability to represent complex sequences effectively. FFN in transformers consists of two linear transformations with a non-linear activation function applied between them. The process begins with the first matrix multiplication, which projects the input Y into a higher-dimensional space using a weight matrix W_1 and adding bias b_1 , as shown in Table II. Next, a non-linear activation function, such as ReLU or GELU is applied. Finally, the second matrix multiplication applies the second linear transformation to the non-linearly transformed output, mapping it back to the original dimensionality.

C. DiP Evaluation

DiP is evaluated against a TPU-like (WS-based) architecture using transformer workloads. Nine widely used transformer models are selected to span various application domains and represent a spectrum of model sizes, from small language models (SLMs) to large language models (LLMs). These models are organized in three types: Encoder-Decoder, Encoder-only, and Decoder-only. Encoder-Decoder models include Vanilla Transformer [10], T5 [30], and BART [31]; Encoder-only models include BERT [35], ALBERT [36], and Transformer-XL [37]; while Decoder-only models include GPT-2 [32], GPT-3 [33], and LLaMA [34]. The models are selected with hyper-parameters to cover a diverse range of workloads. Sequence lengths are chosen from a range of 64 to 2048, specifically (64, 128, 256, 512, 1024, 2048). Additionally, the model's hidden size (d_{model}) varies across (512, 768, 1024, 1280, 5120), while the head size (d_k) is set to either (64, 128). Moreover, the FFN size (d_{FFN}) is configured with values of (2048, 3072, 4096, 5120).

DiP and TPU-like architectures, each with a size of 64×64 , are used for the evaluation. This architecture size aligns well with matrix tiling, as the head size for most transformer models is either 64 or 128. Cycle-accurate simulations are performed to evaluate both DiP and TPU-like implementations in terms of actual latency for each workload and energy consumption. Matrix tiling is used to process matrix multiplication workloads on DiP and TPU-like architectures by dividing the input matrices M_1 and M_2 into sub-matrices (tiles) of 64×64 . By studying many transformer models, the majority of MHA and FFN workload dimensions are divisible by 64. The multiplication is performed per tile as follows; For DiP and TPU-like architectures, every tile of M_2 is loaded once and remains stationary throughout the computation for the

corresponding output tile. For each tile of M_2 , respective tiles from M_1 is iteratively loaded, multiplied, and saved as output psum tiles. After processing all tiles, the final output matrix is constructed by accumulating the associated psum tiles.

Figure 5 compares the energy consumption and latency of DiP and TPU-like 64×64 architectures for MHA and FFN workloads across varying dimensions (M-N-K). DiP consistently outperforms TPU-like implementation for MHA and FFN workloads. Energy improvements for MHA workloads range from 1.81x for smaller workloads to 1.25x for larger ones, while FFN workloads show a similar trend with improvements from 1.8x to 1.25x. These results highlight the energy-efficiency of DiP. Additionally, actual latency for MHA and FFN workloads demonstrates DiP's performance against TPU-like implementation, offering up to 1.49x improvements for smaller workloads, gradually reducing to approximately 1.03x for larger workloads. The breakdown of latency improvement happens for larger workloads as TPU-like architecture hides the latency associated with loading more M_1 tiles per every new M_2 tile. In contrast, for small to medium-sized workloads, TPU-like architectures incur the TFPU penalty of loading M_1 tile associated with each new M_2 tile. Additionally, TPU-like architectures still face the overhead of Input/Output synchronization FIFOs, which impact power consumption, latency, and TFPU. The evaluation highlights DiP as an energy-efficient design, making it a compelling alternative to TPU-like architectures.

D. Comparison With Related Accelerators

Table III compares DiP with Google TPU V4i [25], Groq TSP [38], OPTIMUS [39], DTATrans [40], and Eidetic [41], highlighting DiP's performance and energy efficiency for matrix multiplication acceleration. To ensure a fair and transparent comparison, Table III annotate whether each result corresponds to a system-level or a core-level implementation. Also, the maturity level for each design is annotated as follows: Google TPuv4i and Groq TSP represent post-silicon results; DiP is based on post-layout (post-PnR) implementation; OPTIMUS and DTATrans provide post-synthesis results; and Eidetic is based on modelling results. Since each accelerator is implemented in different technology, the performance metrics are normalized to 22nm using DeepScaleTool [42], [43]. Area and energy efficiencies results are presented before and after scaling to 22nm. In the case of Google TPuv4i, the Matrix Multiply Units (MXUs) occupy approximately 11% of the total chip area, yet deliver nearly the entire performance. While the power consumption of the MXUs alone is not explicitly reported, DiP provides higher area efficiency, achieving 8.192 TOPS/mm² compared to 3.14 TOPS/mm² and 0.157 TOPS/mm² for the Google TPU MXUs at 7nm and scaled 22nm, respectively. DiP architecture features 4,096 MACs in a 64×64 configuration, operating at 1 GHz with INT8 precision using commercial 22nm technology. At a 64×64 size with 4096 PEs, DiP offers peak throughput of 8.192 TOPS, energy efficiency with 9.548 TOPS/W, and delivers an area efficiency (computational density) with 8.192 TOPS/mm². These results demonstrate DiP's capability to provide high

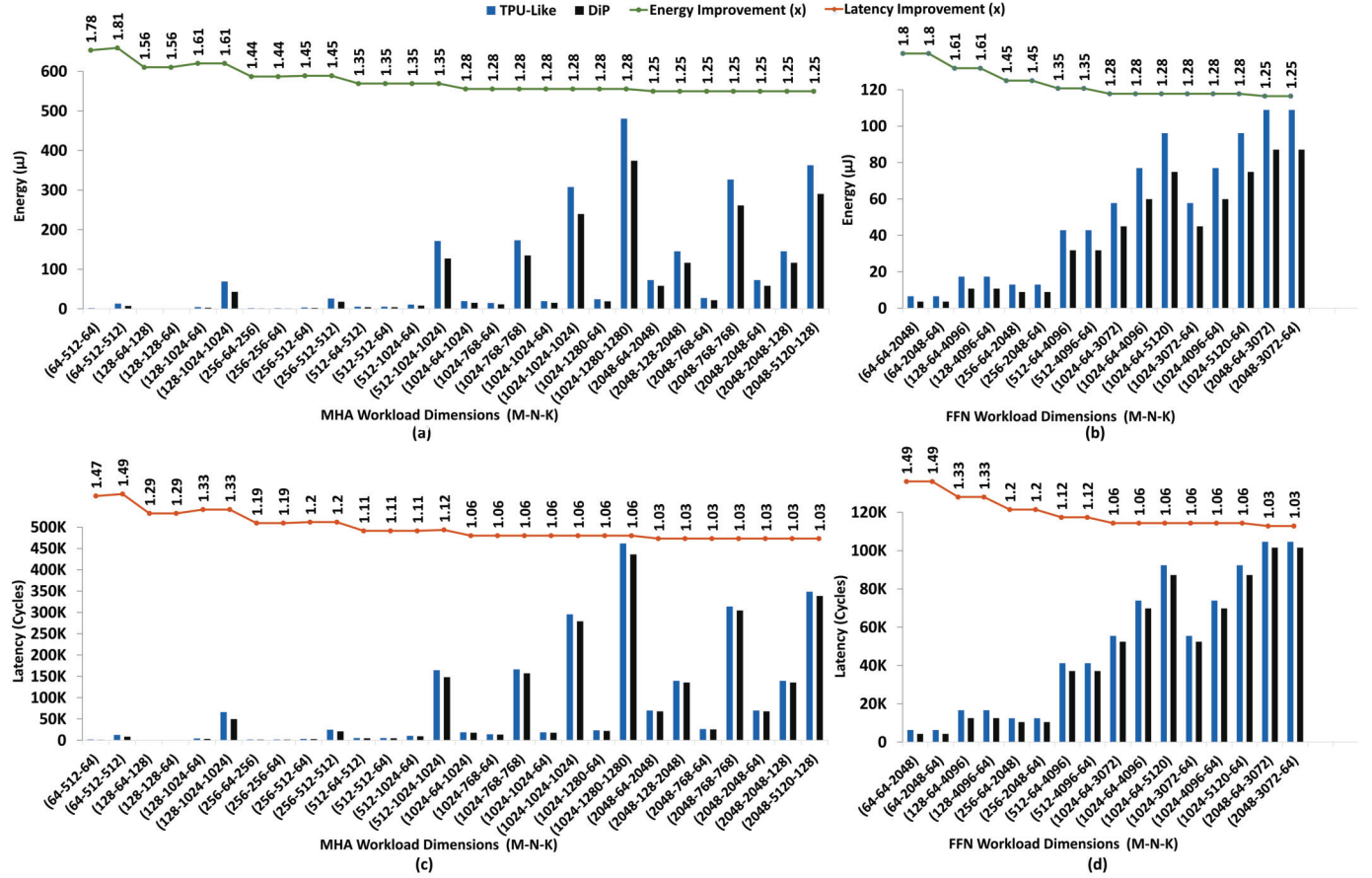


Fig. 5. Evaluation of DiP and TPU-like architecture at size 64×64 with MHA and FFN transformer workloads. The evaluation includes actual energy consumption (a, b) and latency (c, d) across various workloads dimensions of matrix multiplication (M-N-K).

TABLE III
COMPARISON WITH STATE-OF-THE-ART ACCELERATORS

	DiP (this work)	Google TPU V4i [25]	Groq TSP [38]	OPTIMUS [39]	DTATrans [40]	Eidetic [41]
Architecture	64×64 , MACs	$4 \times 128 \times 128$, MACs	Tensor Processors	SA-RCSC	VSSA	In-Memory Computing
Design Level	Core-level, Post-PnR	System-level, Post-Silicon	System-level, Post-Silicon	System-level, Post-Syn	Core-level, Post-Syn	Core-level, Modeling
Frequency (GHz)	1	1.05	0.9	0.2	1	1.5
Precision	INT8	INT8	INT8	INT16	Multiple	Multiple
Technology	22nm	7nm	14nm	28nm	40nm	22nm
Power (W)	0.858	175	300	0.732	0.803	235.7
Area (mm^2)	1	400	725	5.2	1.49	266.4
Peak Throughput (TOPS)	8.192	138	820	0.5	1.304	175.5
Area Efficiency (TOPS/ mm^2)	8.192	0.345	1.131	0.096	0.979	0.659
Energy Efficiency (TOPS/W)	9.548	0.786	2.733	0.683	1.623	0.745
Area Efficiency (TOPS/ mm^2) ⁽¹⁾	8.192	0.017	0.412	0.153	2.984	0.659
Energy Efficiency (TOPS/W) ⁽¹⁾	9.548	0.345	1.823	0.797	2.470	0.745

⁽¹⁾Power and area are normalized to 22nm using DeepScaleTool [42,43]

throughput while minimizing energy consumption, showcasing its compact and optimized design. These metrics make

DiP particularly well-suited for energy-efficient Transformers-based applications.

V. CONCLUSION

In this paper, a diagonal-input and permuted weight-stationary systolic array is proposed to accelerate matrix multiplication. DiP features an architecture of $N \times N$ PEs with a novel dataflow that eliminates the input and output synchronization FIFO buffers required by the conventional WS systolic array. As a result, DiP outperforms WS in terms of throughput, latency, area, and power consumption. Additionally, the analytical models for latency, throughput, FIFO overhead, and TFPU are developed for DiP and WS architectures. The analytical model shows that DiP architecture achieves up to 50% improvement in throughput and TFPU over WS counterparts. Moreover, hardware design space exploration is presented for both DiP and WS architectures using commercial 22nm technology, demonstrating power consumption savings of up to 19.95%, area savings of up to 8.12% at 1 GHz, and energy efficiency per area improvement of up to $2.02\times$. Furthermore, DiP is evaluated using various transformer workloads from widely used models such as GPT-2, GPT-3, BERT, BART, and LLaMA. DiP outperforms TPU-like architecture, achieving energy improvements ranging from $1.25\times$ to $1.81\times$ and latency improvements ranging from $1.03\times$ to $1.49\times$ across various transformer MHA and FFN workloads. DiP achieves a peak performance of 8.192 TOPS with energy efficiency of 9.548 TOPS/W. A comparison between state-of-the-art accelerators and DiP is presented, demonstrating DiP's potential as a promising architecture for energy-efficient matrix multiplication acceleration. This paper is the foundation for DiP architecture and dataflow. Future work will focus on leveraging multiple DiP cores and deploying optimized transformers to enhance energy efficiency and performance.

REFERENCES

- [1] B. Mondal, "Artificial intelligence: State of the art," *Recent Trends Adv. Artif. Intell. Internet Things*, vol. 172, pp. 389–425, May 2020.
- [2] G. G. Chowdhury, "Natural language processing," *Annu. Rev. Inf. Sci. Technol.*, vol. 37, no. 1, pp. 51–89, 2003.
- [3] D. Khurana, A. Koli, K. Khatter, and S. Singh, "Natural language processing: State of the art, current trends and challenges," *Multimedia Tools Appl.*, vol. 82, no. 3, pp. 3713–3744, Jan. 2023.
- [4] A. Gillioz, J. Casas, E. Mugellini, and O. A. Khaled, "Overview of the transformer-based models for NLP tasks," in *Proc. 15th Conf. Comput. Sci. Inf. Syst. (FedCSIS)*, Sep. 2020, pp. 179–183.
- [5] H. Zhang, H. Song, S. Li, M. Zhou, and D. Song, "A survey of controllable text generation using transformer-based pre-trained language models," *ACM Comput. Surv.*, vol. 56, no. 3, pp. 1–37, Mar. 2024.
- [6] Y. Wang et al., "Transformer in action: A comparative study of transformer-based acoustic models for large scale speech recognition applications," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Jun. 2021, pp. 6778–6782.
- [7] P. Xu, X. Zhu, and D. A. Clifton, "Multimodal learning with transformers: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 10, pp. 12113–12132, Oct. 2023.
- [8] K. Han et al., "A survey on vision transformer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 1, pp. 87–110, Jan. 2023.
- [9] T. Lin, Y. Wang, X. Liu, and X. Qiu, "A survey of transformers," *AI Open*, vol. 3, pp. 111–132, Apr. 2022.
- [10] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
- [11] J. Hoffmann et al., "Training compute-optimal large language models," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst.*, Jan. 2024, pp. 1–36.
- [12] L. Floridi and M. Chiriatti, "GPT-3: Its nature, scope, limits and consequences," *Minds Mach.*, vol. 30, no. 4, pp. 681–694, 2020.
- [13] H.-T. Kung, "Why systolic architectures?," *IEEE Comput.*, vol. 15, no. 1, pp. 37–46, Jan. 1982.
- [14] Y. H. Hu and S.-Y. Kung, "Systolic arrays," in *Handbook of Signal Processing Systems*, S. S. Bhattacharyya, E. F. Deprettere, R. Leupers, and J. Takala, Eds., Cham, Switzerland: Springer, 2019, pp. 939–977.
- [15] Z. Yang et al., "Systolic array based accelerator and algorithm mapping for deep learning algorithms," in *Proc. IFIP Int. Conf. Netw. Parallel Comput.*, 2018, pp. 153–158.
- [16] M. Soltaniyeh, R. P. Martin, and S. Nagarakatte, "An accelerator for sparse convolutional neural networks leveraging systolic general matrix-matrix multiplication," *ACM Trans. Archit. Code Optim.*, vol. 19, no. 3, pp. 1–26, Sep. 2022.
- [17] B. Wang, S. Ma, G. Zhu, X. Yi, and R. Xu, "A novel systolic array processor with dynamic dataflows," *Integration*, vol. 85, pp. 42–47, Jul. 2022.
- [18] H. Waris, C. Wang, W. Liu, and F. Lombardi, "AxSA: On the design of high-performance and power-efficient approximate systolic arrays for matrix multiplication," *J. Signal Process. Syst.*, vol. 93, no. 6, pp. 605–615, Jun. 2021.
- [19] G. Shomron, T. Horowitz, and U. Weiser, "SMT-SA: Simultaneous multithreading in systolic arrays," *IEEE Comput. Archit. Lett.*, vol. 18, no. 2, pp. 99–102, Jul. 2019.
- [20] M. A. Hanif, R. V. W. Putra, M. Tanvir, R. Hafiz, S. Rehman, and M. Shafique, "MPNA: A massively-parallel neural array accelerator with dataflow optimization for convolutional neural networks," 2018, *arXiv:1810.12910*.
- [21] C. Peltekis, D. Filippas, G. Dimitrakopoulos, C. Nicopoulos, and D. Pnevmatikatos, "ArrayFlex: A systolic array architecture with configurable transparent pipelining," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Apr. 2023, pp. 1–6.
- [22] J. Dean, D. Patterson, and C. Young, "A new golden age in computer architecture: Empowering the machine-learning revolution," *IEEE Micro*, vol. 38, no. 2, pp. 21–29, Mar. 2018.
- [23] N. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Architect.*, 2017, pp. 1–12.
- [24] N. P. Jouppi et al., "A domain-specific supercomputer for training deep neural networks," *Commun. ACM*, vol. 63, no. 7, pp. 67–79, Jun. 2020.
- [25] N. P. Jouppi et al., "Ten lessons from three generations shaped Google's TPUv4: Industrial product," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, Valencia, Spain, 2021, pp. 1–14, doi: [10.1109/ISCA52012.2021.00010](https://doi.org/10.1109/ISCA52012.2021.00010).
- [26] B. Asgari, R. Hadidi, and H. Kim, "MEISSA: Multiplying matrices efficiently in a scalable systolic architecture," in *Proc. IEEE 38th Int. Conf. Comput. Design (ICCD)*, Oct. 2020, pp. 130–137.
- [27] R. Xu, S. Ma, Y. Guo, and D. Li, "A survey of design and optimization for systolic array-based DNN accelerators," *ACM Comput. Surv.*, vol. 56, no. 1, pp. 1–37, Jan. 2024.
- [28] E. Yago, P. Castelló, S. Petit, M. E. Gómez, and J. Sahuquillo, "Impact of the array shape and memory bandwidth on the execution time of CNN systolic arrays," in *Proc. 23rd Euromicro Conf. Digit. Syst. Design (DSD)*, Aug. 2020, pp. 510–517.
- [29] Y. H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, Apr. 2016, pp. 367–379.
- [30] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, Jan. 2020.
- [31] M. Lewis et al., "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," 2019, *arXiv:1910.13461*.
- [32] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [33] T. B. Brown et al., "Language models are few-shot learners," 2020, *arXiv:2005.14165*.
- [34] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [35] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [36] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soicrut, "ALBERT: A lite BERT for self-supervised learning of language representations," 2019, *arXiv:1909.11942*.
- [37] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, "Transformer-XL: Attentive language models beyond a fixed-length context," 2019, *arXiv:1901.02860*.

- [38] D. Abts et al., “Think fast: A tensor streaming processor (TSP) for accelerating deep learning workloads,” in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit. (ISCA)*, May 2020, pp. 145–158.
- [39] J. Park, H. Yoon, D. Ahn, J. Choi, and J. Kim, “OPTIMUS: Optimized matrix multiplication structure for transformer neural network accelerator,” *Proc. Mach. Learn. Syst.*, vol. 2, pp. 363–378, Mar. 2020.
- [40] T. Yang et al., “DTATrans: Leveraging dynamic token-based quantization with accuracy compensation mechanism for efficient transformer architecture,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 2, pp. 509–520, Feb. 2023.
- [41] C. Eckert, A. Subramanian, X. Wang, C. Augustine, R. Iyer, and R. Das, “Eidetic: An in-memory matrix multiplication accelerator for neural networks,” *IEEE Trans. Comput.*, vol. 72, no. 6, pp. 1539–1553, Jun. 2023.
- [42] S. Sarangi and B. Baas, “DeepScaleTool: A tool for the accurate estimation of technology scaling in the deep-submicron era,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2021, pp. 1–5.
- [43] DeepScale tool, SourceForge, *DeepScale Tool*. Accessed: Jan. 20, 2025. [Online]. Available: <https://sourceforge.net/projects/deepscaletool/>



Ahmed J. Abdelmaksoud (Member, IEEE) received the B.Sc. and M.Sc. degrees in electronics engineering from Cairo University, Egypt, in 2018 and 2022, respectively. He is currently pursuing the Ph.D. degree with the Centre for Electronics Frontiers (CEF), The University of Edinburgh, U.K. Since 2018, he has been actively involved in digital ASIC design projects across both research and industry. His professional experience includes working as a Research Associate at the System-on-Chip Center, Khalifa University, United

Arab Emirates; an ASIC Physical Design Engineer at Si-Vision, Egypt; and a Research Assistant at the Opto-Nano Electronics Laboratory, Egypt. In addition, his current research interests primarily focus on developing spatial and specialized architectures for efficient AI hardware acceleration.



Shady Agwa (Member, IEEE) received the B.Sc. and M.Sc. degrees in electrical engineering from Assiut University, Egypt, and the Ph.D. degree in electronics engineering from The American University in Cairo, Egypt, in 2018. Following the Ph.D. degree, he joined the Computer Systems Laboratory, Cornell University, USA, as a Post-Doctoral Associate for two years. In 2021, he joined the Centre for Electronics Frontiers, University of Southampton, U.K., as a Senior Research Fellow and then as a Research Fellow at The University of Edinburgh, U.K. He is currently a Research Fellow with the Centre for Electronics Frontiers (CEF), The University of Edinburgh, U.K. His research interests include VLSI and computer architecture for AI using conventional and emerging technologies. His work focuses on ASIC-driven AI architectures with extensive expertise in in-memory computing, stochastic computing, systolic arrays, beyond Von Neumann architectures, memories, and energy-efficient digital ASIC design.



Themis Prodromakis (Senior Member, IEEE) received the bachelor's degree in electrical and electronic engineering from the University of Lincoln, U.K., the M.Sc. degree in microelectronics and telecommunications from the University of Liverpool, U.K., and the Ph.D. degree in electrical and electronic engineering from Imperial College London, U.K. He then held a Corrigan Fellowship in nanoscale technology and science with the Centre for Bio-Inspired Technology, Imperial College London, and a Lindemann Trust Visiting Fellowship with the Department of Electrical Engineering and Computer Sciences, University of California at Berkeley, USA. He was a Professor of nanotechnology at the University of Southampton, U.K. He holds the Regius Chair of Engineering at The University of Edinburgh and is the Director of the Centre for Electronics Frontiers. He is currently the Royal Academy of Engineering Chair in emerging technologies and a Royal Society Industry Fellowship. His background is in electron devices and nanofabrication techniques. His current research interests include memristive technologies for advanced computing architectures and biomedical applications. He is a fellow of the Royal Society of Chemistry, the British Computer Society, IET, and the Institute of Physics.